



Prism

A real-time, multi-modal underwriting engine for new-to-credit and thin-file customers

Cash-flow and behavioural signals · an interpretable scorecard · exact reason codes · guardrailed AI explanations

Ansh Dixit · September 2026



Creditworthy, but unscorable

Traditional models rely on static bureau history - which millions of good borrowers simply do not have yet

25%

of applicants have no bureau file

thin file: another 20%

0%

NTC approval under a bureau-only model

"no hit" = auto-decline

43.8%

overall approval, bureau-only

at a 6% portfolio bad rate

0.45

age adverse-impact ratio

fails the 4/5ths rule

What it has to do

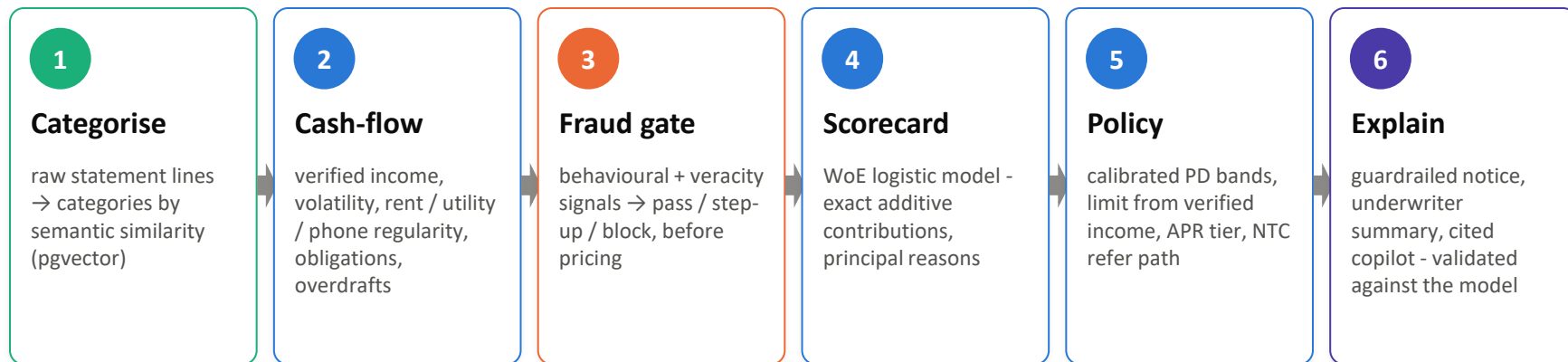
- Expand access for New-to-Credit and thin-file customers with alternative data and real-time behavioural signals
- Mitigate fraud at the same time - not as an afterthought
- Regulatory transparency: every decision explainable and defensible
- Shift from reactive scoring to proactive, contextual decisioning

Why it is hard

- No file means no signal - unless you bring a modality the applicant actually has: their cash flow
- Signals that catch fraud must never leak into price (fair lending)
- Reg B requires specific principal reasons; a black box cannot supply them and an LLM must not invent them
- Real-time: the answer is expected before the applicant leaves the page

The approach in one line

A scorecard and rules make the decision. The language model only explains and retrieves - and is checked before it speaks.



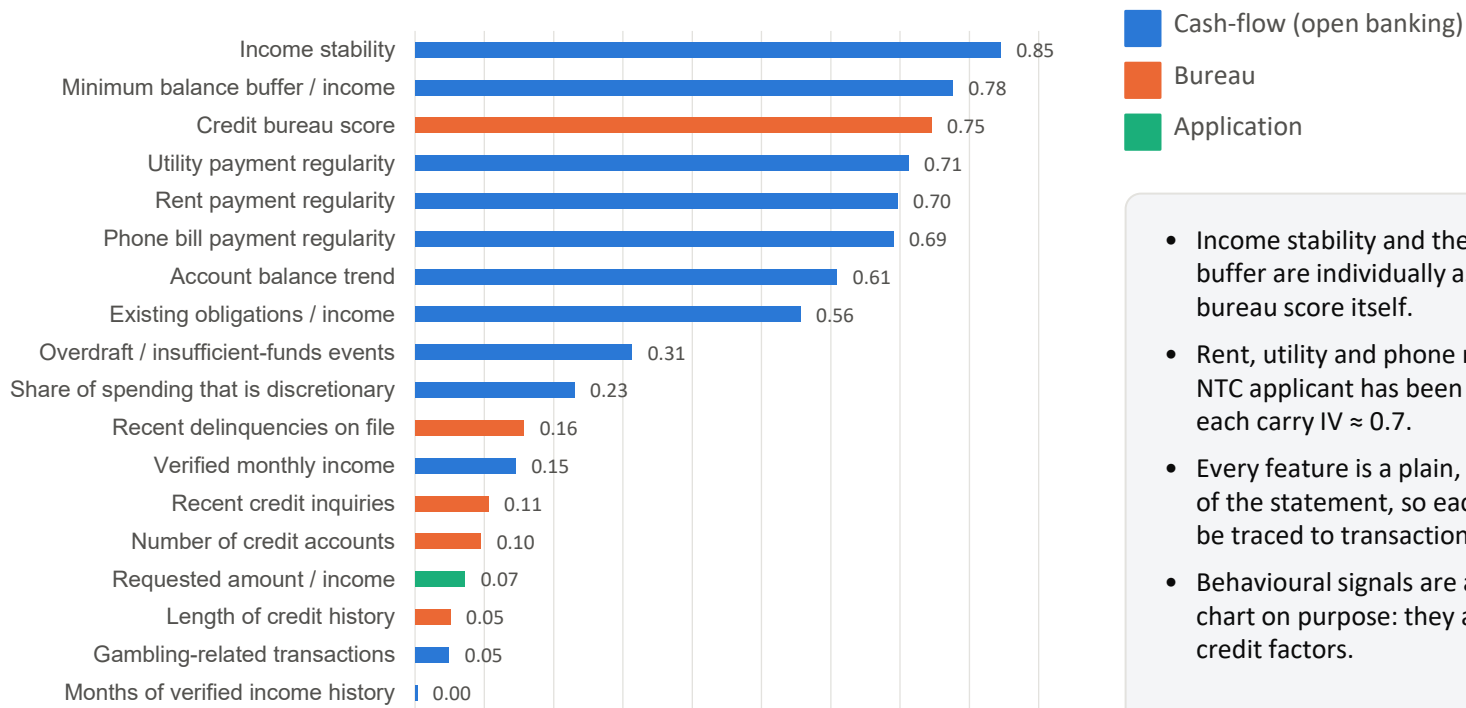
≈ 100 ms end to end before the LLM step, on one request

Three modalities, one scorecard, one invariant

Bureau data when it exists · consented bank cash-flow data · behavioural signals captured live in the form. An NTC applicant with a linked account scores on the same card as everyone else; without one they are referred, never auto-declined. Behavioural signals feed the fraud gate only - they never change a legitimate applicant's price.

Insight 1 - cash flow is as informative as the bureau

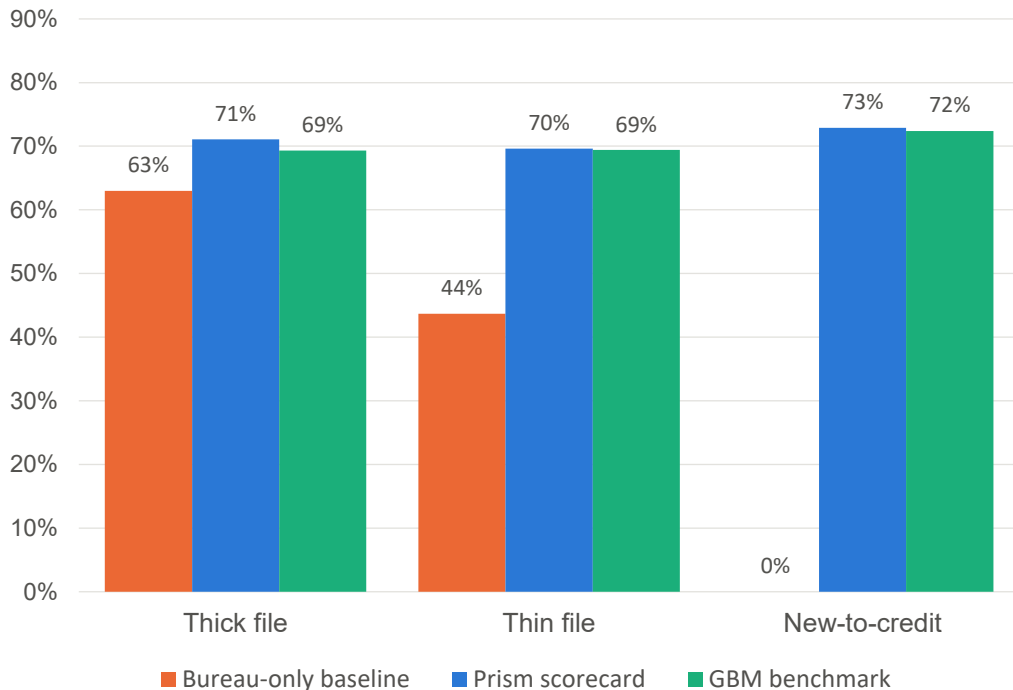
Information value per feature on the hold-out set (higher = more predictive of 12-month delinquency)



- Income stability and the minimum-balance buffer are individually as informative as the bureau score itself.
- Rent, utility and phone regularity - things an NTC applicant has been doing for years - each carry IV ≈ 0.7 .
- Every feature is a plain, explainable statistic of the statement, so each contribution can be traced to transactions.
- Behavioural signals are absent from this chart on purpose: they are fraud signals, not credit factors.

Insight 2 - more approvals at exactly the same risk

Approval rate by file type when every model is cut off at a 6% realised bad rate on the approved book (hold-out)



43.8% → 71.3%

overall approval rate, same 6% bad rate

0% → 73%

New-to-credit approvals

realised bad rate 7.0%

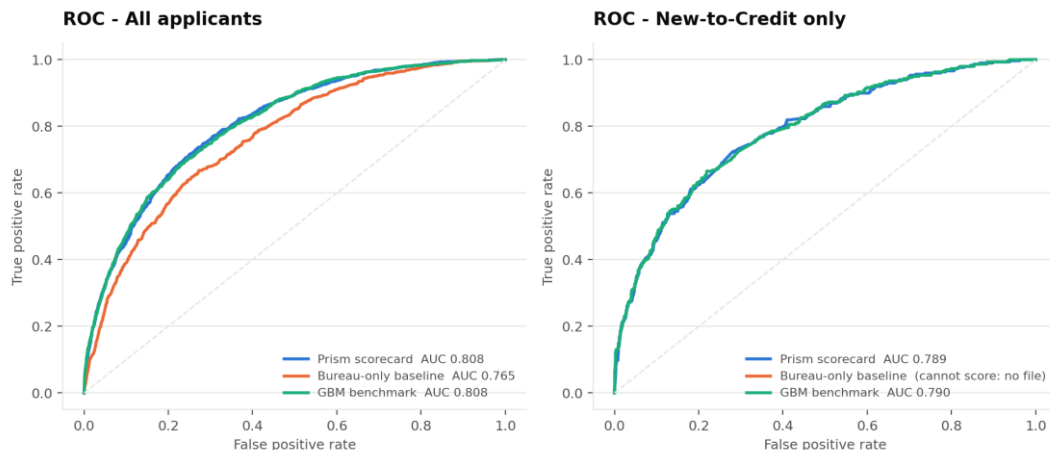
29.8%

of applicants newly approved ("swap-in")

they default at 7.4% · 60% are NTC

Insight 3 - the interpretable model costs nothing here

ROC on the hold-out set: the WoE scorecard vs the bureau-only baseline vs a gradient-boosting benchmark



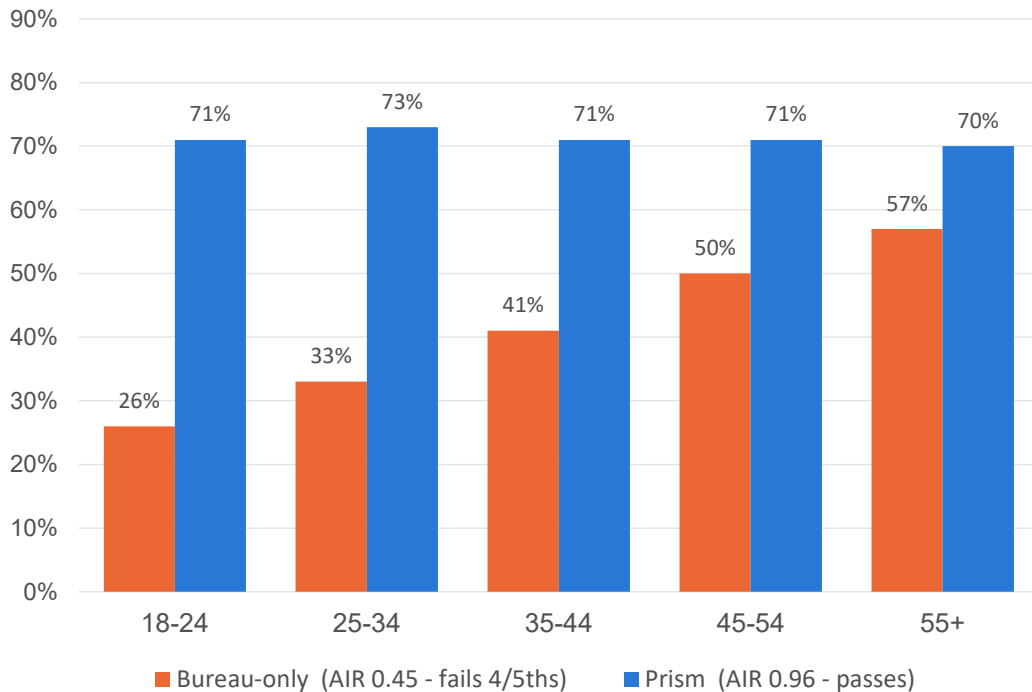
Why a scorecard

- Additive by construction: $\text{logit} = \text{intercept} + \sum \text{contributions}$. Reason codes are exact, not a SHAP approximation.
- Regulators, model-risk teams and underwriters already know how to validate it.
- The GBM benchmark quantifies what interpretability costs - here AUC 0.809 vs 0.808.
- Honest caveat: the synthetic generator is additive in its latent factor, which flatters linear models. On real data expect a 0.01–0.03 AUC gap; the measurement, not the number, is the point.

Segment	Bureau-only	Prism scorecard	GBM benchmark
All applicants	0.7653	0.8085	0.8081
Thick file	0.7895	0.8131	0.8109
Thin file	0.6908	0.8241	0.8247
New-to-credit	cannot score	0.789	0.7902

Insight 4 - inclusion is measurable, so measure it

Approval rate by age band. Age is never a model input; it is used only to test the model.



0.45 → 0.96

adverse-impact ratio (age)

4/5ths rule threshold = 0.80

34% → 3%

equal-opportunity gap

max - min TPR among good payers

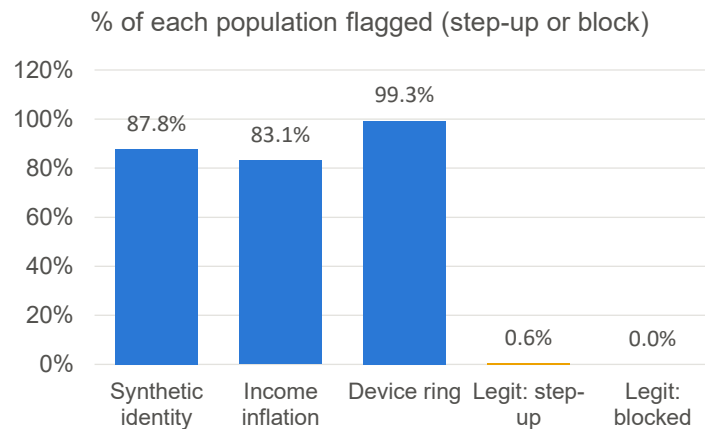
Why it moves: young applicants are disproportionately NTC. A bureau-only lender turns "no history yet" into "declined"; cash-flow evidence gives them a way to prove themselves. Sex and region AIR ≥ 0.95 under both models.

The fraud & verification gate runs before pricing

Additive rule points on behavioural, device and veracity signals. Two thresholds. A block needs at least two strong signals.

Rule	Signal	Points
F01	Stated income > 1.35× bank-verified	40
F02	Identity number pasted into the form	25
F03	Device used for ≥ 3 applications / 30 days	35
F04	Email address younger than 60 days	15
F05	VoIP phone number	10
F06	Application completed in < 2 minutes	15
F07	Income field edited ≥ 3 times	10
F08	Income unverifiable - no linked account	20

≥ 60 points → BLOCK (decline, fraud review) · 35–59 → STEP-UP (verify identity / income first) · < 35 → PASS



99.4%

block precision

recall 35%

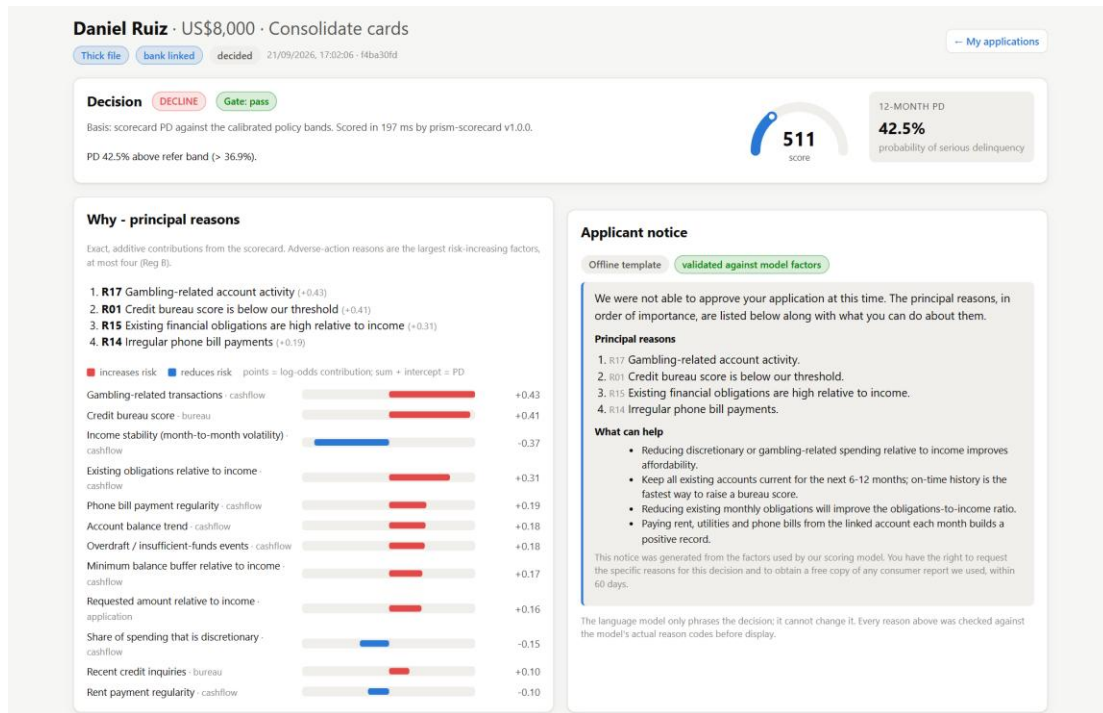
0.59%

friction on legitimate

step-up rate

Explainability that survives a regulator

Exact reasons from the model; a notice the language model may phrase but cannot bend



1 Contributions → principal reasons

Largest risk-increasing factors, at most four, aggregated by reason code. R01–R18 map to Reg B principal-reason language.

2 Absence of evidence ≠ bad evidence

A no-file applicant gets R19 "no credit bureau file", never "score below threshold". Missing data has its own codes.

3 The LLM is checked before it speaks

Its notice must restate the decision and exactly the model's codes, with no protected-class or promissory language - or it is rejected and a deterministic template is issued.

Architecture

React · Spring Boot 4 · PostgreSQL 16 + pgvector · Amazon Bedrock (or Anthropic API / offline) · AWS ECS + RDS

Prism - real-time multi-modal underwriting engine

Scorecard + rules decide; the LLM only explains and retrieves. Every model call is guardrailed, validated against the model's own reason codes, and audited.

FRONTEND

React 19 · Vite · TS

Applicant & underwriter UI

Apply form with live behavioural capture
(session timing, device events, device fingerprint)
Decision view - queue - copilot - model card
JWT in session storage - role-based routes

CLOUD (AWS)

ECS Fargate · ALB + WAF · RDS Postgres 16

Task IAM role - no keys in containers
Secrets Manager: JWT secret, DB password
VPC endpoints to Bedrock / Secrets / Logs
CloudWatch logs + Actuator metrics, alarms on latency, LLM fallback rate, validation failures

Local: docker compose + mvnw + vite

Offline AI providers - runs with zero credentials

SECURITY

Defence in depth

HS256 JWT · Spring Security resource server
Method-level RBAC: applicant / underwriter / admin
Bean Validation on every request field
Token-bucket rate limit · CORS allow-list · CSP
SSN stored only as keyed HMAC · PII redaction before any prompt - problem-detail errors, no leaks
Secrets from env only - startup fails without them

BACKEND · Spring Boot 4 / Java 21

REST API /api/auth · /api/applications · /api/copilot · /api/policy · /api/model · /api/monitoring

Real-time decision pipeline (one request, ~100 ms before the LLM step)



GuardrailService

the only path to any LLM - identical for every provider



Retrieval (pgvector, cosine, HNSW)

category_exemplars - transaction categorisation
historical_applicants - "applicants like this one" (advisory)
policy_chunks - cited policy RAG for the copilot
Embeddings: Titan v2 (1024-d) or offline hashing, same schema

Human in the loop & audit

decisions: model version, feature snapshot, every contribution, reason codes, notice provenance
underwriter_actions: confirm / override with reason
Tests: unit + Python-parity + Testcontainers flow

DATA · PostgreSQL 16 + pgvector (Flyway-managed)

Structured

users · applications (SSN as HMAC only) · bank_transactions
decisions (append-only) · underwriter_actions · ai_invocations

Vector (1024-d, cosine, HNSW index)

category_exemplars · historical_applicants · policy_chunks · applications.profile_embedding

Seeded at start-up per embedding provider; re-embedded automatically when the provider changes.

AI LAYER

provider = env var

Amazon Bedrock (primary)

Claude via Converse API (model id config)
Titan Text Embeddings v2 (1024-d)
Bedrock Guardrails: PII, denied topics, prompt-attack filter, grounding

Anthropic API (alternate)

Same prompts, validator and audit trail

Offline (fallback + zero-credential demo)

Deterministic template renderer built from the same context the validator checks
Hashed n-gram embeddings (lexical)

Three closed tasks, versioned prompts

ADVERSE_ACTION_NOTICE - Reg B wording
UNDERWRITER_SUMMARY - cites factor codes
COPILOT_ANSWER - cites retrieved policy only

Never decides, prices, ranks or overrides.

↑ artefacts packaged with the service

ML PIPELINE · Python (offline)

ml/generate_data.py → train_and_export.py

30k synthetic applicants (thick / thin / NTC)
WoE scorecard vs bureau-only vs GBM benchmark
Policy simulation at 6 % portfolio bad rate
Fairness harness (AIR, equal opportunity)
Fraud-gate evaluation · model card

Exports: scorecard.json, fraud_rules.json, model_card.json, historical_sample.json, figures

Design invariants

1. Behavioural signals feed the fraud gate only - they never change the price or limit offered to a legitimate applicant.
2. Reason codes are exact: the scorecard is additive, and a notice that name change the model did not produce is rejected before it can reach a customer.
3. No-fine applicants are referred, never auto-declined; with a linked account they score on the same card as everyone else.
4. Every decision is reproducible from its stored record (model version, feature snapshot, contributions) without re-running the model.

AI layer - one guarded path to any model

Provider is an environment variable. Prompts are versioned files. Every call is validated and audited.

1

PII redaction

ids, emails, phones,
cards masked before
anything leaves

2

Managed guardrail

Bedrock ApplyGuardrail
on input & output (PII,
denied topics, prompt
attacks)

3

Model call

Bedrock Converse ·
Anthropic API · offline
template, with a hard
timeout

4

Output validator

JSON schema · reason
codes $\subseteq$ model's ·
decision restated · no
protected-class terms

5

Retry then fallback

one corrective retry,
then the deterministic
template

6

Audit row

provider, model, latency,
tokens, validation,
fallback, prompt hash

Providers

- Amazon Bedrock: Claude via Converse, Titan Text Embeddings v2 (1024-d), Bedrock Guardrails
- Anthropic API: same prompts, validator, audit
- Offline: template engine + hashed n-gram embeddings - zero credentials

Three closed tasks

- ADVERSE_ACTION_NOTICE - plain-language, Reg B wording
- UNDERWRITER_SUMMARY - strengths / risks citing factor codes
- COPILOT_ANSWER - retrieval-orchestrated, cites only retrieved policy sections

What the model never does

- Decide, price, rank or override
- See a name, contact detail or identifier
- Cite policy it was not given
- Reach a customer unvalidated

Data layer - PostgreSQL 16 + pgvector

One database for structured decision records and three vector stores, schema managed by Flyway

Structured

- applications - identity minimised; the national ID is stored only as a keyed HMAC for duplicate / synthetic-identity checks
- bank_transactions - each line with its assigned category and match confidence
- decisions - append-only: model version, feature snapshot, every contribution, reason codes, notice provenance, latency
- underwriter_actions - confirm / request docs / override, with reason
- ai_invocations - the audit trail of every model call
- users - bcrypt hashes, three roles

Vector (1024-d, cosine, HNSW index)

- category_exemplars - 104 phrases across 21 categories; a raw description takes its nearest exemplar's category
- historical_applicants - 800 decided cases with known outcomes; "applicants like this one" for the underwriter (advisory only)
- policy_chunks - internal policy documents split by section; the copilot may cite only what it retrieved
- applications.profile_embedding - every live case becomes searchable
- Same schema for Titan v2 and the offline embedder; stores re-seed automatically when the provider changes

Security layer

Basics done properly, because a credit system is a target



Authentication

HS256 JWT issued at login, verified by Spring Security resource-server support; stateless, 8-hour expiry



Authorisation

Method-level roles: applicant (own cases), underwriter, admin. Applicants cannot read others' cases or the AI summary



Input validation

Bean Validation on every field with bounds; consent enforced before bank data is processed



Secure API usage

Rate limiting on login / submit / copilot, CORS allow-list, CSP, RFC 9457 errors that never leak internals



No hardcoded secrets

JWT secret and DB password from environment only; start-up fails without them; no default credential anywhere



Responsible data handling

SSN stored as keyed HMAC; PII redacted before any prompt; prompts logged by hash only; AWS access via IAM roles

The prototype, end to end

Six synthetic personas exercise every path: approve, refer, decline, step-up, block, and the no-file inclusion path

The 'Prism Underwriting' persona loader screen displays a 'Start with a demo persona' section with three options: 'Priya - New to Credit gig worker', 'Alex Morgan - synthetic identity', and 'Ramesh - High file, irregular income'. The 'Priya' option is selected. Below this, the 'Applicant' section shows details for Priya Raman, including her email, phone, and employment status. The 'Bureau pull' section shows a simulated bureau pull result with a score of 636 and a reason code of R19. The 'Linked account' section shows a simulated bank pull result with a balance of \$12,450. The 'Behavioral signals' section shows a simulated behavioral signal result.

Apply - persona loader, consent, simulated bureau pull and bank feed, live behavioural capture

The 'Prism Underwriting' decision screen displays a 'Decision' section with a '636' score and a '0.9%' risk level. The 'Why - principal reasons' section lists three reasons: '1. R19 Low minimum account balance relative to income', '2. R19 No credit history for a significant credit history to evaluate', and '3. R19 No credit history for a significant credit history to evaluate'. The 'Applicant notice' section provides a summary of the decision and the reasons for it.

Decision - Priya (no file, gig income) approved in ~100 ms; reason R19 = "no bureau file", not "low score"

The 'Prism Underwriting' block screen displays a 'Decision' section with a '637' score and a '0.9%' risk level. The 'Before decision can be reached' section lists three reasons: '1. R19 Number of recent credit inquiries', '2. R19 Number of recent credit inquiries', and '3. R19 Number of recent credit inquiries'. The 'Applicant notice' section provides a summary of the decision and the reasons for it.

Block - a synthetic identity with a spotless statement is stopped before pricing

The 'Prism Underwriting' underwriting queue screen displays a table of underwriting applications. The table has columns for 'SUBMITTED', 'APPLICANT', 'FILE', 'REQUESTED', 'DECISION', 'DATE', 'SCORE', 'PD', and 'STATUS'. The table shows three applications: 'JAN2024_1710147', 'JAN2024_1710148', and 'JAN2024_1710149'. The 'DECISION' column shows 'APPROVED', 'DECLINED', and 'APPROVED' respectively.

Queue - underwriter view with decision, gate outcome, score and PD per case

The 'Prism Underwriting' underwriter summary screen displays a 'Underwriter summary' section with a '636 (PD 0.9%)' score and a '0.9%' risk level. The 'Underwriting copilot' section provides a summary of the decision and the reasons for it. The 'Record action' section shows a simulated record action result.

Underwriter - AI summary on demand, override with a logged reason, copilot with policy citations

The 'Prism Underwriting' model & fairness screen displays a 'Prism Multi-Modal Underwriting Scorecard' section with a '636' score and a '0.9%' risk level. The 'DISCRIMINATION (HOLD-OUT AVE)' section shows a simulated discrimination result. The 'POLICY SIMULATION' section shows a simulated policy simulation result. The 'DECISION RANGES (CALCULATED)' section shows a simulated decision range result.

Model & fairness - performance by segment, policy simulation, AIR by group, live monitoring

Code walkthrough

Clean, modular packages; the scoring engine is a pure function, parity-tested against the Python export

```
ScorecardEngine.score(features)
double logit = scorecard.intercept();
for (Feature f : scorecard.features()) {
    Double v = features.get(f.name());
    boolean missing = v == null || v.isNaN();
    double woe = f.woeFor(v);    // missing bin, not
    imputed
    double c = f.coefficient() * woe;
    logit += c;
    contributions.add(new FeatureContribution(
        f.name(), f.label(), f.modality(),
        f.reasonCode(),
        missing ? null : v, missing, woe, c));
}
double pd = 1 / (1 + Math.exp(-logit));
int score = toScore(logit);    // 600 = 30:1 odds, PDO
20
return new ScoreResult(...,
    principalReasons(contribs));

// principalReasons: aggregate c > 0 by reason code,
// unobserved features → missing-data code (R19/R20),
// drop noise below the floor, keep the top four.
```

backend/ packages

scoring · fraud · cashflow · ai (bedrock, anthropic, offline, validator, guardrail, audit) · explain · similar · knowledge · copilot · application · auth · model · monitoring · common

Records for immutable values, constructor injection, no Lombok, interfaces at every provider boundary (LlmClient, EmbeddingClient, TextGuardrail, TransactionCategorizer).

Tests

- 34 unit tests: engine, policy, fraud gate, cash-flow arithmetic, validator, redaction, offline template
- Parity: Java reproduces Python's score, PD and reason codes on 300 hold-out rows
- Testcontainers flow on real pgvector: login → submit → decision → summary → copilot → action; role denials

Engineering practices

What a reviewer can verify in the repository

1

API-first

docs/api/openapi.yaml; every endpoint validated and role-guarded; RFC 9457 errors

3

README & run

One env file, docker compose for pgvector, mvnw and npm scripts; full stack also runs as containers

5

Testing

Unit + parity + Testcontainers integration; TypeScript strict build; Playwright walkthrough for screenshots

7

Configuration

Typed @ConfigurationProperties with validation; provider swaps are one variable

2

Version control

Git history with milestone commits; .gitattributes, .gitignore, .env.example - the secret file is never committed

4

Architecture diagram

docs/architecture.svg (source) + PNG; deployment topology in infra/aws

6

Credentials

Environment only; start-up fails fast without a 32-char JWT secret; AWS via IAM roles; Secrets Manager in deployment

8

Reproducibility

Seeded synthetic data, exported artefacts versioned with the service, model version on every decision row

Responsible AI, and what is not yet proven

The model card ships with the service (/api/model/card); its limitations are part of the deliverable

Built in

- Protected characteristics are never inputs; fairness (AIR, equal opportunity) tested before release and reported in the card
- Every decision reproducible from its stored record without re-running the model
- The LLM explains only; its output is validated against the model's reason codes; fallback is deterministic
- Referred cases always reach a person; overrides are recorded with reasons and feed retraining
- Data minimisation: hashed identifiers, redacted prompts, hash-only logging
- Monitoring plan: score stability, realised vs predicted bad rate, AIR by group, override rate, LLM validation / fallback rates

Limitations, stated plainly

- Synthetic data: the method is real, the absolute numbers are not production performance
- Bureau pull and bank connection are simulated behind adapter interfaces
- Fairness tested on three proxies; production needs BISG-style analysis and ongoing monitoring
- The scorecard is linear in WoE space; interactions are not modelled (the GBM benchmark measures the cost)
- The offline mode is a template renderer, not a language model - the live path needs Bedrock or an API key
- Cash-flow features need consented bank linking; unlinked applicants are referred, which adds friction



From reactive scoring to proactive, contextual decisioning

44% → 71%

Approval

at the same 6% bad rate

0% → 73%

New-to-credit

scored on the same card

0.45 → 0.96

Age AIR

passes the 4/5ths rule

exact

Explainability

reasons validated before a customer sees them

Next: server-side bureau and open-banking adapters · customer IdP · model registry with the card attached · BISG fairness monitoring · Bedrock in production with the managed guardrail

Repository: [README](#) with setup, tests and deployment · Ansh Dixit · anshbaldixit@gmail.com